

[← Go Back](#)

Learn

[Arduino Ecosystem](#) >[Microcontrollers](#) >[Digital Pins](#)[Analog Input Pins](#)[Basics of PWM \(Pulse Width Modulation\)](#)[Debugging Fundamentals](#)[Guide to 3V3 and 5V Power Supplies Differences](#)[Programming](#) >[Electronics](#) >[Communication](#) >[Hardware Design](#) >[Built-in Libraries](#) >[Contributions](#) >[Edge AI](#) >[Home](#) / [Learn](#) / [Digital Pins](#)

Digital Pins

Discover how digital pins work and how they can be configured.

Last revision 09.02.2026

The pins on the Arduino can be configured as either inputs or outputs. This document explains the functioning of the pins in those modes. While the title of this document refers to digital pins, it is important to note that the vast majority of Arduino (ATmega) analog pins may be configured, and used, in exactly the same manner as digital pins.

Properties of Pins Configured as INPUT

Arduino (ATmega) pins default to inputs, so they don't need to be explicitly declared as inputs with `pinMode()` when you're using them as inputs. Pins configured this way are said to be in a **high-impedance state**. Input pins make extremely small demands on the circuit that they are sampling, equivalent to a series resistor of 100 MΩ in front of the pin. This means that it takes very little current to move the input pin from one state to another, and can make the pins useful for

ON THIS PAGE

Properties of Pins Configure as INPUT

[Pull-up Resistors with Pins Configured as INPUT](#)[Properties of Pins Configure as INPUT_PULLUP](#)[Properties of Pins Configure as OUTPUT](#)[Hilfe](#)

Digital Pins | Arduino Documentation
and can make the pins useful for
such tasks as implementing a

capacitive touch sensor,
reading an LED as a photodiode,
or reading an analog sensor with
a scheme such as RCTime.

This also means however, that
pins configured as
`pinMode(pin, INPUT)` with
nothing connected to them, or
with wires connected to them
that are not connected to other
circuits, will report seemingly
random changes in pin state,
picking up electrical noise from
the environment, or capacitively
coupling the state of a nearby
pin.

Pull-up Resistors with Pins Configured as INPUT

Often it is useful to steer an
input pin to a known state if no
input is present. This can be
done by adding a pull-up resistor
(to +5 VDC), or a pull-down
resistor (resistor to GND) on the
input. A 10 k Ω resistor is a good
value for a pull-up or pull-down
resistor.

Properties of Pins Configured as INPUT_PULLUP

There are 20 k Ω pull-up resistors

Digital Pins | Arduino Documentation
built into the ATmega chip that
can be accessed from software.

These built-in pull-up resistors
are accessed by setting

`pinMode()` to `INPUT_PULLUP`.

This effectively inverts the
behavior of the `INPUT` mode,
where `HIGH` means the sensor
is off, and `LOW` means the
sensor is on.

The value of this pull-up
depends on the microcontroller
used. On most AVR-based
boards, the value is guaranteed
to be between 20 kΩ and 50 kΩ.
On the Arduino Due, it is
between 50 kΩ and 150 kΩ. For
the exact value, consult the
datasheet of the microcontroller
on your board.

When connecting a sensor to a
pin configured with
`INPUT_PULLUP`, the other end
should be connected to ground.
In the case of a simple switch,
this causes the pin to read
`HIGH` when the switch is open,
and `LOW` when the switch is
pressed.

The pull-up resistors provide
enough current to dimly light an
LED connected to a pin that has
been configured as an input. If
LEDs in a project seem to be
working, but very dimly, this is
likely what is going on.

Pull-up Resistors and Pin Mode Switching

On AVR-based Arduino boards,
the pull-up resistors are

controlled by the same registers
(internal chip memory locations)

that control whether a pin is

`HIGH` or `LOW`. At the hardware register level, a pin configured as `INPUT_PULLUP` shares the same PORT register bit as a pin set to `OUTPUT HIGH`. Consequently, a pin that has its pull-up resistor enabled while in `INPUT_PULLUP` mode will be driven `HIGH` if it is then switched to `OUTPUT` mode via `pinMode()`, because the underlying PORT register bit remains set.

However, the reverse is not true when using the Arduino API:

calling `pinMode(pin, INPUT)` explicitly clears both the direction register (DDR) and the output register (PORT), which disables the internal pull-up resistor regardless of the pin's previous state. This means that an output pin left in a `HIGH` state will **not** retain its pull-up resistor when switched to `INPUT` mode through `pinMode()`. If you need the internal pull-up enabled, use `INPUT_PULLUP`:

```
1 pinMode(pin, INPUT_PUL
```

Prior to Arduino 1.0.1, it was possible to configure the internal pull-ups in the following manner:

```
1 pinMode(pin, INPUT);  
2 digitalWrite(pin, HIGH);
```

NOTE: Digital pin 13 is harder to use as a digital input than the other digital pins because it has an LED and resistor attached to it that's soldered to the board on most boards. If you enable its internal 20 kΩ pull-up resistor, it will hang at around +1.7 VDC instead of the expected +5 VDC because the onboard LED and series resistor pull the voltage level down, meaning it always returns `LOW`. If you must use pin 13 as a digital input, set its `pinMode()` to `INPUT` and use an external pull-down resistor.

Properties of Pins Configured as OUTPUT

Pins configured as `OUTPUT` with `pinMode()` are said to be in a **low-impedance state**. This means that they can provide a substantial amount of current to other circuits. ATmega pins can source (provide positive current) or sink (provide negative current) up to 40 mA (milliamps) of current to other devices/circuits. This is enough current to brightly light up an LED (don't forget the series resistor), or run many sensors, for example, but not enough

current to run most relays, solenoids, or motors.

Short circuits on Arduino pins, or attempting to run high current devices from them, can damage or destroy the output transistors in the pin, or damage the entire ATmega chip. Often this will result in a "dead" pin in the microcontroller but the remaining chip will still function adequately. For this reason it is a good idea to connect `OUTPUT` pins to other devices with 470 Ω or 1 k Ω resistors, unless maximum current draw from the pins is required for a particular application.

See Also

[pinMode\(\)](#)

[digitalWrite\(\)](#)

[digitalRead\(\)](#)

Suggest changes	Need support?	License & Trademark
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license. Arduino and the Arduino logo are trademarks or registered trademarks of Arduino S.r.l.

Was this article helpful?



ARDUINO®

 [Back to top](#)

© 2026 Arduino

[Trademarks & Copyrights](#)

[Whistleblowing](#)

[Digital Services Act](#)

[Terms of Service](#)

[Privacy Policy](#)

[Security](#)

[Cookie Settings](#)